



Building with SQLite.

Yesterday you learned what a database IS. Today we build one — ALTER, CRUD, Streamlit integration, and the expense tracker rebuilt with real persistence. The storage pattern every group will use in Phase 4.

PROGRAM

Python Internship

LED BY

Zlaark

SESSION

10 of 25

How we'll spend our 90 minutes today.

Yesterday was theory. Today is building. We take the SQL you learned and put it into a real Streamlit app — with the expense tracker as the capstone.

DURATION
90 min

00:00 – 00:10



Day 9 Part 1 Recap & Q&A

JSON, database fundamentals, SQL queries (`SELECT` , `WHERE` , `CREATE TABLE` , `JOIN`). Quick check.

00:10 – 00:20



ALTER TABLE + DROP TABLE

Change a table after it exists. Add columns, rename, delete.

00:20 – 00:35



SQLite Deep-Dive + CRUD

The full pattern: connect→CREATE→INSERT→commit. Then `SELECT` , `UPDATE` , `DELETE` .

00:35 – 00:50



Inspecting the DB + Streamlit Pattern

Three tools to look inside a `.db` file. The helper-function pattern for Streamlit.

00:50 – 01:00



Context Manager + pandas Bridge

The `with` block (auto-close). `pd.read_sql_query` and `df.to_sql` .

01:00 – 01:30



Exercise: Expense Tracker with SQLite

Rebuild Day 8's app with a real database. `CREATE TABLE` , `INSERT` on submit, `SELECT` to display.

Yesterday you learned the WHAT. Today we do the HOW.

Quick check that yesterday's SELECT exercise worked. If it didn't, we fix it now — today builds directly on SELECT, WHERE, and CREATE TABLE.

WHAT WE COVERED YESTERDAY DAY 09 • PART 1

- ✓ **JSON** — dicts to files, `json.dump` / `load`
- ✓ **What a database is** — the spreadsheet analogy, tables/rows/columns
- ✓ **SQL fundamentals** — `SELECT`, `WHERE`, the cursor pattern
- ✓ **CREATE TABLE** — columns, types, constraints (`NOT NULL`, `DEFAULT`, `PRIMARY KEY`)
- ✓ **Relationships** — `FOREIGN KEY`, `JOIN`, aggregates (`COUNT`, `SUM`, `AVG`)

CHECK YOUR LAPTOP NOW VERIFY

```
# You should have yesterday's exercise:
$ ls day9/
queries.py
expenses.db

# Run it — should print 5 query results:
$ python queries.py
# → All expenses (20 rows)
# → Food expenses (8 rows)
# → Unique categories (5)
# → 5 most recent expenses
# → Total spent: ₹ 12,450
```

⚠ **IF expenses.db IS MISSING** — download it from the shared folder.
If `queries.py` didn't work, raise your hand. Today's exercise needs you to be comfortable with `SELECT` and `WHERE`.

ALTER TABLE — change a table after it exists. DROP TABLE — delete it.

You defined the table with CREATE TABLE. But what if you forgot a column? What if you need to rename one? What if the table is wrong and you want to start over? These are the **structure-changing** commands.

```

alter_drop.sql
SQL

-- === ALTER TABLE: add a column ===
ALTER TABLE expenses ADD COLUMN payment_method TEXT DEFAULT 'Cash';
-- → adds a new column to every existing row
-- → existing rows get the DEFAULT value ('Cash')

-- === ALTER TABLE: rename a column (SQLite 3.25+) ===
ALTER TABLE expenses RENAME COLUMN note TO description;
-- → changes the column name

-- === ALTER TABLE: rename the whole table ===
ALTER TABLE expenses RENAME TO expense_entries;
-- → the table is now called expense_entries

-- === DROP TABLE: delete the whole table ===
DROP TABLE expenses;
-- → deletes the table AND all its data. Permanent.
-- → there is no undo. Be very sure.

-- === Safe pattern: DROP IF EXISTS ===
DROP TABLE IF EXISTS old_expenses;
-- → only drops if the table exists. No error if it doesn't.
```

WHEN TO USE 3 SCENARIOS

-  **You forgot a column** — You created the table, started adding data, then realized you need a `payment_method` column. Don't recreate the table — `ALTER TABLE ADD COLUMN`. Your existing data is safe.
-  **You need to rename** — A column name is confusing. `ALTER TABLE RENAME COLUMN`. Your data stays, only the name changes.
-  **You want to start over** — The table structure is completely wrong. `DROP TABLE` removes it entirely. Then CREATE TABLE with the correct structure. **BUT — all data is lost.** Export first if you need it.

BACKUP BEFORE DROP

If your table has data you care about, `SELECT *` into a backup before `DROP`. There is **no undo** in SQL. Day 8's CSV was just a file — you could copy it. A database table is not a file you can copy with `cp`.

- 1 ALTER TABLE ADD COLUMN** — Adds a column to an existing table. Existing rows get the DEFAULT value (or NULL if no default). No data is lost.
- 2 DROP TABLE is permanent** — It deletes the table AND every row in it. There is no recycle bin, no undo. Always `DROP TABLE IF EXISTS` to avoid errors.
- 3 Cross-reference: Day 6** — Like `df['new_col'] = ...` in pandas (adding a column) or `del df['col']` (removing one). But here the change is permanent — it modifies the database file.

SQLite — a full SQL database. In one file. No server.

SQLite is a database that lives in a single `.db` file on your laptop. Python's `sqlite3` module is built in — no pip install. **CREATE TABLE**, **INSERT**, **SELECT**. The same SQL every database uses.

```

sqlite_demo.py PYTHON

import sqlite3

# === 1. Connect to (or create) a database ===
conn = sqlite3.connect("expenses.db")
cursor = conn.cursor()

# === 2. Create a table ===
cursor.execute("""
    CREATE TABLE IF NOT EXISTS expenses (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        amount REAL,
        category TEXT,
        date TEXT,
        note TEXT
    )
""")

# === 3. Insert a row ===
cursor.execute("""
    INSERT INTO expenses (amount, category, date, note)

```

- 1 `sqlite3.connect("file.db")` — Opens a connection to the database file. Creates it if it doesn't exist. Returns a connection object.
- 2 `cursor.execute(SQL)` — Runs a SQL statement. CREATE TABLE defines the structure. INSERT adds a row. SELECT reads rows.
- 3 The `?` placeholders — Use `?` for values, NEVER string formatting. This prevents SQL injection (a real security issue). The values go in a tuple as the second argument.
- 4 `conn.commit() + conn.close()` — `commit()` saves the changes. `close()` releases the connection. Forgetting `commit()` means your data is lost. Forgetting `close()` leaks resources.

THE MENTAL MODEL CONCEPT

-  **Tables, rows, columns**
Like a spreadsheet, but structured. Each table has named columns with types (`TEXT`, `INTEGER`, `REAL`).
-  **PRIMARY KEY**
A unique ID for each row. `AUTOINCREMENT` means SQLite assigns the next number automatically.
-  **Persistent**
Once committed, the data survives app restarts, laptop reboots, crashes. It's in the file.
-  **Fast**
SQLite handles millions of rows. Queries use indexes. No need to load the whole file into memory.

KEY IDEA `SQLite is a real database, not a file format.` You write SQL queries, not file operations. The learning curve is SQL, not Python — but the basics (CREATE, INSERT, SELECT) take 10 minutes.

Four operations. CREATE, READ, UPDATE, DELETE. The whole database API.

Every database app does these four things. Once you know them, you can build any CRUD app — expense tracker, note app, attendance system, contact list.

SELECT



Read rows from a table

```
# All rows
cursor.execute("SELECT * FROM expenses")
rows = cursor.fetchall()

# Filter with WHERE
cursor.execute(
    "SELECT * FROM expenses WHERE category = ?",
    ("Food",)
```

`fetchall()` returns a list of tuples. Each tuple is a row. Use **WHERE** to filter.

INSERT



Add a new row

```
cursor.execute("""
    INSERT INTO expenses
    (amount, category, date, note)
    VALUES (?, ?, ?, ?)
""", (450, "Food", "2026-08-05", "Lunch"))
conn.commit()
```

Use `?` for values. **Always commit()** after INSERT — otherwise the row is lost.

UPDATE



Modify an existing row

```
cursor.execute("""
    UPDATE expenses
    SET amount = ?, note = ?
    WHERE id = ?
""", (500, "Lunch (updated)", 3))
conn.commit()
```

Always use WHERE with UPDATE. Without it, you update EVERY row. Always commit().

DELETE



Remove a row

```
cursor.execute(
    "DELETE FROM expenses WHERE id = ?",
    (3,)
)
conn.commit()
```

Always use WHERE with DELETE. Without it, you delete EVERY row. This is the most dangerous SQL command.

THE CRUD PATTERN

Every database app: **Create (INSERT)**, **Read (SELECT)**, **Update (UPDATE)**, **Delete (DELETE)**. Master these four and you can build any data app.

Your database is a file. You can look inside it.

When you run `sqlite3.connect("expenses.db")`, a file appears. But how do you SEE what's in it without writing Python? **Three tools — pick one.**



GUI APP

DB Browser for SQLite

A free GUI app. **Open any .db file**, browse tables, run queries with a click.

INSTALL

Download from `sqlitebrowser.org`

BEST FOR

Beginners. Visual inspection. No command line needed.



VSCODE EXT

VS Code SQLite Extension

An extension right inside VS Code. **Click the .db file**, see the tables.

INSTALL

VS Code → Extensions → search `"SQLite"` → install by `alexcvzz`

BEST FOR

Staying in your editor. Quick look without switching apps.



CLI TOOL

sqlite3 CLI

The command-line tool. **Runs SQL directly.** Built into Python.

USE

`sqlite3 expenses.db` then `SELECT * FROM expenses;`

BEST FOR

Quick checks. Scripts. Servers without a GUI.

TRY IT NOW

Open your `expenses.db` in any of these tools after today's exercise. See the table structure, the rows you inserted, run a query. **The database stops being a black box.**

Streamlit re-runs everything. So open the connection **inside** the function.

Every widget interaction re-runs your whole script top to bottom. If you open a database connection at the top of **app.py**, it opens and closes on EVERY rerun — leaking connections and breaking your app. The fix: a helper function.

THE WRONG WAY



```
# x BAD - connection at the top
import sqlite3, streamlit as st

conn = sqlite3.connect("expenses.db")
# This opens a NEW connection every
# time Streamlit re-runs the script
# (which is every widget interaction)

# ... rest of app ...

conn.close()
# This may never run if the user
# interacts before the script ends
```

Leaks connections. Can lock the database. Breaks on the 3rd or 4th interaction.

THE RIGHT WAY



```
# ✓ GOOD - helper function
import sqlite3, streamlit as st

def get_expenses():
    conn = sqlite3.connect("expenses.db")
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM expenses")
    rows = cursor.fetchall()
    conn.close() # always close!
    return rows

def add_expense(amount, category, date, note):
    conn = sqlite3.connect("expenses.db")
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO expenses VALUES (NULL,?,?,?,?)",
        (amount, category, date, note)
    )
    conn.commit()
    conn.close()

# Now call these from your widgets:
expenses = get_expenses()
st.dataframe(expenses)
```

Each function opens, does its job, closes. No leaks. Works with the run-loop.

THE PATTERN Wrap every database operation in a function. The function opens the connection, does the work, commits (if writing), and closes. Streamlit's run-loop handles the rest.

The **with** block — what it does (and doesn't) do.

Slide 23 (SQLite in Streamlit) showed the helper-function pattern: **open** → **work** → **close**. The `with` block is cleaner — but there's a **common misconception**: `with sqlite3.connect(...)` commits/rolls back the transaction, but it does **NOT** close the connection. Here's the real pattern.

OLD WAY Manual open / close

OPEN → WORK → CLOSE

```

manual_close.py
PYTHON

import sqlite3

def get_expenses():
    conn = sqlite3.connect("expenses.db")
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM expenses")
    rows = cursor.fetchall()
    conn.close() # ← if you forget this, leak
    return rows

# Problems:
# - If cursor.execute() raises an error,
#   conn.close() never runs. Leak.
# - You must remember to close in EVERY function.
# - Easy to forget. Hard to debug.
    
```

Works, but **fragile**. One forgotten `close()` = a leaked connection that locks the file.

CLEAN WAY with block auto-close

OPEN → WORK → AUTO-CLOSE

```

with_block.py
PYTHON

from contextlib import closing
import sqlite3

# Step 1: connect
conn = sqlite3.connect("expenses.db")

# Step 2: use `with conn` for transactions
# This commits on success, rolls back on error.
# But it does NOT close the connection!
with conn:
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM expenses")
    rows = cursor.fetchall()
    # ← transaction committed here
    # But conn is STILL OPEN. Still usable.

# Step 3: actually close the connection
conn.close()
# OR use closing() for true auto-close:
# with closing(sqlite3.connect(...)) as conn:
#     ... # closes when block exits
    
```

`with conn` commits/rolls back the **transaction** — but does **NOT** close the connection. Use `closing()` or explicit `conn.close()` for that.

THE GOTCHA

`with sqlite3.connect(...)` manages the **transaction** (commit on success, rollback on error). It does **NOT** close the connection. For true auto-close, use `with closing(sqlite3.connect(...))` from `contextlib`. Or just call `conn.close()` explicitly — the helper-function pattern from slide 23 still works fine.

pandas reads SQLite directly. No manual fetching.

You already know pandas (Day 6). You just learned SQLite. The bridge is one function: `pd.read_sql_query()`. Write SQL, get a DataFrame. No loops, no fetchall, no manual conversion.

```

pandas_sqlite.py PYTHON

import pandas as pd
import sqlite3

# === Read: SQL → DataFrame ===
conn = sqlite3.connect("expenses.db")
df = pd.read_sql_query("SELECT * FROM expenses", conn)
conn.close()

# df is a DataFrame. You know what to do with it.
print(df.head())
print(df["amount"].mean()) # average — pandas, not SQL

# === Write: DataFrame → SQL ===
new_data = pd.DataFrame({
    "amount": [500, 300],
    "category": ["Food", "Transport"],
    "date": ["2026-08-05", "2026-08-05"],
    "note": ["Dinner", "Auto"]
})
new_data.to_sql("expenses", conn, if_exists="append", index=False)
# → inserts both rows in one call

```

- 1 `pd.read_sql_query(sql, conn)` — Runs the SQL, returns a DataFrame. No fetchall, no loop.
- 2 `df.to_sql(table, conn, if_exists="append")` — Inserts a whole DataFrame in one call. Faster than row-by-row INSERT.
- 3 `if_exists="append"` — Critical. Default is `"fail"` (error if table exists). Use `"append"` to add rows, `"replace"` to wipe and rewrite.

WITHOUT PANDAS VS WITH PANDAS COMPARE

✗ WITHOUT PANDAS (MANUAL)

```

cursor.execute("SELECT * FROM expenses")
rows = cursor.fetchall()
# rows is a list of tuples
# Convert to list of dicts manually?
# Or use cursor.description for column names?
# Then build a DataFrame with
# pd.DataFrame(rows, columns=...)
# 5 lines of boilerplate

```

✓ WITH PANDAS (ONE LINE)

```

df = pd.read_sql_query("SELECT * FROM expenses", conn)
# Done. DataFrame with proper column names.

```

THE PATTERN

Use **pandas for reading** (SELECT). Use **raw SQL for writing** (INSERT/UPDATE/DELETE).
pandas + SQL = the best of both worlds.

SQL errors are confusing. Until they're not.

Five errors you'll hit today. Read the error, find it here, fix it. **The error message is telling you exactly what's wrong** — you just need to learn the vocabulary.

01

❗ OPERATIONALERROR

```
sqlite3.OperationalError: no such table: expenses
```

🔍 CAUSE

You're querying a table that doesn't exist yet. You forgot **CREATE TABLE**, or you misspelled the table name.

🔧 FIX

Run `CREATE TABLE IF NOT EXISTS expenses (...)` first. Check the spelling — SQL is case-insensitive but spelling matters.

02

❗ OPERATIONALERROR

```
sqlite3.OperationalError: near 'WHERE': syntax error
```

🔍 CAUSE

SQL syntax error. Usually a **missing comma, quote, or parenthesis** before the WHERE.

🔧 FIX

Read the SQL string carefully. The error points to WHERE, but the actual mistake is usually on the **line BEFORE** it.

03

❗ INTEGRITYERROR

```
sqlite3.IntegrityError: NOT NULL constraint failed
```

🔍 CAUSE

You tried to INSERT a row with a **NULL (empty) value** into a column marked NOT NULL.

🔧 FIX

Make sure every required field has a value before INSERT. Or remove the **NOT NULL** constraint if the field is optional.

04

❗ PROGRAMMINGERROR

```
sqlite3.ProgrammingError: Error binding parameter — wrong count or type
```

🔍 CAUSE

Two causes: **(1) Wrong count** — 4 `?` placeholders but only 3 values. Raises `ProgrammingError`. **(2) Wrong type** — passing a string where SQLite expects a number. Raises `InterfaceError` in Python < 3.13, `ProgrammingError` in Python 3.13+.

🔧 FIX

Count the `?` in your SQL. Count the values in your tuple. **They must match exactly**. Also check types — don't pass a list where SQLite expects a single value.

05

❗ PROGRAMMINGERROR

```
sqlite3.ProgrammingError: Cannot operate on a closed database
```

🔍 CAUSE

You called `conn.close()` but then tried to use the **cursor or connection again**.

🔧 FIX

Move `conn.close()` to the very end of the function. Or use `from contextlib import closing` and `with closing(sqlite3.connect(...)) as conn:` for true auto-close.

THE RULE Read the **error TYPE** (OperationalError, IntegrityError, etc). It tells you the category. Then read the **MESSAGE** — it names the exact problem. Google the exact error string if still stuck.

Bookmark this. Three storage formats, three use cases.

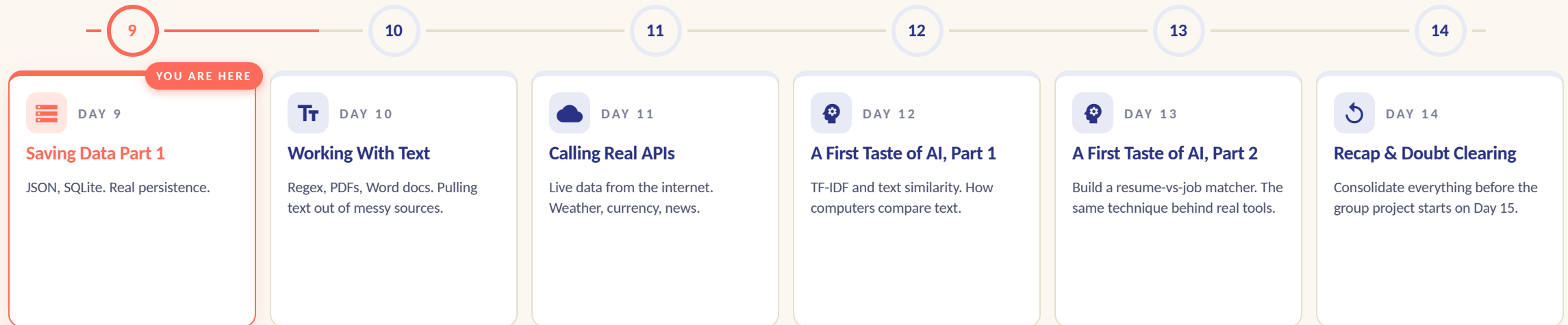
The skill is not knowing the API — it's knowing which format fits the data. Here's the decision tree.

 CSV FLAT • TABULAR GOOD FOR Flat tabular data. Expenses, student records, logs. Easy to inspect in Excel. NOT FOR Nested data, relationships, concurrent writes. CODE<pre>pd.read_csv("data.csv") df.to_csv("data.csv", index=False)</pre> PICK WHEN Your data is a flat table and you want to inspect it by hand.	 JSON NESTED • CONFIG GOOD FOR Config files, settings, nested data. API responses (Day 11). Small + read-mostly. NOT FOR Large datasets (loads entire file into memory), querying/filtering, multiple writers. CODE<pre>json.dump(obj, f) json.load(f)</pre> PICK WHEN Your data is nested or structured, and you read/write the whole thing at once.	 SQLite RELATIONAL • SQL GOOD FOR Anything with relationships, queries, or 100+ rows. The storage every production app uses. NOT FOR Simple config — JSON is easier. Read-only data that never changes. CODE<pre>conn = sqlite3.connect("app.db") cursor.execute("SELECT * FROM ...")</pre> PICK WHEN You need to query, filter, or relate data. Or your dataset is growing.
--	--	---

DECISION TREE Flat table you can inspect by hand? → **CSV**. Nested data or config? → **JSON**. Need to query, filter, or relate? → **SQLite**. When in doubt, start with CSV — upgrade to SQLite when you outgrow it.

Six days. From storage to a first taste of AI.

Today is **Day 2 of Phase 3**. Here's where it goes — storage, text, APIs, and the AI technique behind resume matchers.



THE ARC Days 9-11 add the skills. Days 12-13 add the AI. Day 14 consolidates. Day 15, the group project begins.

PART 3 OF 3 • CLOSING EXERCISE

03 / 03

Upgrade Day 8's app. From CSV to SQLite.

Today's exercise takes the expense tracker you built on Day 8 and replaces the CSV storage with a real SQLite database. Same app, real persistence, the pattern every group will use in Phase 4.

1 BRIEF • 5 MIN

›

2 BUILD • 15 MIN

›

3 EXTEND • 5 MIN

Replace CSV with SQLite. Same app, real database.

Take Day 8's expense tracker (or the scaffold below if you don't have yours). Replace the CSV storage with SQLite. `CREATE TABLE` , `INSERT` on submit, `SELECT` to display.

THE BRIEF

TASK

Build a Streamlit app called `expense_sqlite.py` that stores expenses in a SQLite database instead of a CSV file. Same UI as Day 8 — form to add, metric for total, chart, table. Different storage.

REQUIREMENTS

- ✓ Create an `expenses` table with columns: id, amount, category, date, note
- ✓ On form submit: `INSERT` a row into the table
- ✓ On app load: `SELECT` all rows and display in `st.dataframe`
- ✓ Compute the total with `SELECT SUM(amount) FROM expenses`
- ✓ Use the helper-function pattern from slide 8 (open → work → close)
- ✓ Use `?` placeholders for all values (SQL injection prevention)

BY THE END

Every student has an expense tracker that persists data in a real SQLite database. The Phase-4 storage pattern.

STARTING SCAFFOLD

`expense_sqlite.py`

```
# expense_sqlite.py – starting scaffold
import streamlit as st
import sqlite3

st.title("Expense Tracker (SQLite)")

# TODO: create the table if it doesn't exist
def init_db():
    conn = sqlite3.connect("expenses.db")
    conn.execute("""
        CREATE TABLE IF NOT EXISTS expenses (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            amount REAL, category TEXT,
            date TEXT, note TEXT
        )
    """)
    conn.commit()
    conn.close()

# TODO: helper to add an expense
```

RUN IT `streamlit run expense_sqlite.py` . Add an expense, restart the app, confirm it persisted. Check `expenses.db` with a SQLite viewer if you have one.

One way to solve it. There are others — this is just clean.

Walk through this line by line. Then go back to your own version and compare.

expense_sqlite.py

FULL SOLUTION

FUNCTION DEFINITIONS

```
import streamlit as st
import sqlite3
import pandas as pd
```

```
① def init_db():
    conn = sqlite3.connect("expenses.db")
    conn.execute("""
        CREATE TABLE IF NOT EXISTS expenses (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            amount REAL, category TEXT,
            date TEXT, note TEXT
        )
    """)
    conn.commit()
    conn.close()

② def add_expense(amount, category, date, note):
    conn = sqlite3.connect("expenses.db")
    conn.execute(
        "INSERT INTO expenses (amount, category, date, note) VALUES (?, ?, ?, ?)",
        (amount, category, date, note)
    )
    conn.commit()
    conn.close()

③ def get_expenses():
    conn = sqlite3.connect("expenses.db")
    df = pd.read_sql_query(
        "SELECT * FROM expenses ORDER BY id DESC", conn
    )
    conn.close()
    return df
```

```
④ def get_total():
    conn = sqlite3.connect("expenses.db")
    df = pd.read_sql_query(
        "SELECT SUM(amount) FROM expenses", conn
    )
    conn.close()
    return df.fetchone()[0] or 0.0
```

MAIN PROGRAM • UI

```
⑤ init_db()
st.title("Expense Tracker (SQLite)")

with st.form("add_expense", clear_on_submit=True):
    amount = st.number_input("Amount", min_value=0.0)
    category = st.selectbox("Category",
        ["Food", "Transport", "Books", "Other"], key="cat")
    date = st.date_input("Date").isoformat()
    note = st.text_input("Note", key="note")
    submitted = st.form_submit_button("Add")

⑥ if submitted:
    add_expense(amount, category, date, note)
    st.success("Expense added!")

⑦ total = get_total()
st.metric("Total Spent", f"₹ {total:,.2f}")
df = get_expenses()
st.dataframe(df)
```

LINE BY LINE

7 STEPS

- 1 init_db() — create the table**
Runs once on startup. `CREATE TABLE IF NOT EXISTS` is idempotent — safe to call every rerun.
- 2 add_expense() — INSERT**
Opens connection, inserts with `?` placeholders, commits, closes. The helper-function pattern from slide 8.
- 3 get_expenses() — SELECT**
Uses `pd.read_sql_query` to load directly into a DataFrame. `ORDER BY id DESC` shows newest first.
- 4 get_total() — SUM**
SQL aggregation. `fetchone()` returns a tuple. `or 0.0` handles the empty-table case (SUM returns None).
- 5 init_db() + form**
Call `init_db()` at startup. The form uses `clear_on_submit=True` to empty the fields after adding. All widgets keyed.
- 6 On submit — call add_expense**
Standard `st.form_submit_button` pattern. Call the helper, show a success message.
- 7 Display — metric + table**
`get_total()` for the headline, `get_expenses()` for the detail. Same UI as Day 8, different storage.

THE UPGRADE — Same UI as Day 8. Different storage. **CSV → SQLite**. This is the pattern every group will use in Phase 4 — start with CSV, upgrade to SQLite when you need queries.

```
    ).fetchone()
DAY 09 • SAVING DATA
    return result[0] or 0.0
```


Finished early? Add a delete feature. Or peek at tomorrow.

Three extension ideas for fast students. Plus a preview of Day 10 — Working With Text.

EXTENSION IDEAS

FOR FAST STUDENTS

1

Add a delete button

Next to each expense in the table, add a way to delete it. `DELETE FROM expenses WHERE id = ?`. You'll need to pass the row's `id`. (Hint: use `st.button` with a unique key per row.)

2

Add a category filter

In the sidebar, add a selectbox for category. Filter the table: `SELECT * FROM expenses WHERE category = ?`. Now users can view just Food, or just Transport.

3

Add a chart with SQL aggregation

Use `SELECT category, SUM(amount) FROM expenses GROUP BY category` to get totals per category. Load into a DataFrame, pass to `st.bar_chart`. SQL does the aggregation — pandas just displays it.

WHAT'S NEXT • DAY 10

WORKING WITH TEXT

Tt

Regex basics

Find patterns in text. Phone numbers, emails, dates. Python's `re` module.

PDF

Reading PDFs and Word docs

Pull text out of PDFs (`PyPDF2`) and `.docx` files (`python-docx`). The foundation for resume matching.

Q

Extracting contact info

A real exercise: extract name, email, phone from a sample resume.

TOMORROW — Your apps learn to read. Text out of PDFs, patterns out of strings. The foundation for the AI exercises on Days 12–13.

IF YOU'RE STUCK — Pair with a neighbour. Read the error message out loud. Raise your hand. We move at the room's pace.

Your app now has a real database.

Tomorrow we make it read — text out of PDFs, patterns out of strings. The foundation for the AI exercises later this week.

OPEN THE FLOOR

ASK NOW

- ✓ "Which storage format felt most natural — JSON, CSV, or SQLite?"
- ✓ "Did the helper-function pattern (open → work → close) click?"
- ✓ "Anything from the exercise that didn't work?"

BETWEEN SESSIONS

GET UNSTUCK

- ✓ **Email** • hello@zlaark.com
For SQLite questions, paste your code + the exact error message.
- ✓ **Web** • www.zlaark.com
Program updates and resources.
- ✓ **In-session** • **raise a hand**
No question is too small. We move at the room's pace.

WHAT'S NEXT • DAYS 10-14

PHASE 3 CONTINUES

- ✓ **Day 11** — Working With Text. Regex, PDFs, Word docs.
- ✓ **Day 12** — Calling Real APIs. Live data from the internet.
- ✓ **Days 12-13** — A first taste of AI. TF-IDF, similarity matching.